BadaGPT電信:思考入力の全体把握に基づくクロスプラットフォーム・アプリケーション生成

— BADA グラビトン電信の拡張としての設計・実装・評価 —

山口 雅旭 (Masaaki Yamaguchi) Global Differential Manifold Research · masaaki-avnturle/Bada

2026 年 10 月 12 日 · BadaGPT電信 v1.0.0

要旨

本稿は、利用者が思い描いたアプリケーションを自然言語で依頼すると、その**依頼の全体**を捉えてアプリケーションを生成し、検証し、生成過程を論文にまとめ、Android (APK)・Windows 10/11・Linux 向けに配布できる形にするシステム「BadaGPT電信」を報告する。BadaGPT電信は、Bada 言語で駆動される通信アプリ「BADA グラビトン電信」の電信エンジン（Bada インタプリタ、ガンマ・ベータ関数による多様体不変量、Jones 多項式、FSK 音響モデム）を受け継ぎ、その上に「全体把握 → 設計 → 生成 → 検証 → 論文」のパイプラインを載せたものである。

中心となる考え方は、依頼文を節に分け、**すべての節**を仕様のいずれかの要素へ対応付け、対応付けられなかった節を捨てずに成果物と論文の双方へ明記することである。これを「節の網羅率」として定量化した。14 種のアプリ部品を組み合わせて依存ゼロの単一 HTML アプリを生成し、構文・オフライン性・依頼部品の実装などを機械的に検証する。

評価では、規則の開発に用いた 24 件の依頼でモジュール選択 F1 100%、規則の修正を終えた後に書いた**未見の 10 件**で F1 81.5%・完全一致率 70% を得た。未見の誤りの事例でも網羅率は 100% であり、網羅率は「取りこぼしがないこと」を示すが「正しく解釈したこと」は示さないことを確認した。生成した全 34 件のアプリは静的検証に合格し、実ブラウザで 7 件の依頼から生成したアプリの全部品を操作して実行時エラー 0 件であった。

キーワード: 要求の全体把握, 節網羅率, アプリケーション自動生成, 自己完結 HTML, クロスプラットフォーム配布, Bada 言語, FSK 音響通信

1. 序論

1.1 背景

Bada は、量子的な演算子（左作用 <-・多様体積分 -<・右量子作用 >-）と名前空間命令を持つプログラミング言語であり、本リポジトリではこれを用いた多数のアプリケーションが作られてきた。そのひとつ「BADA グラビトン電信」は、宛先とメッセージを Bada プログラム `telegraph.bada` で符号化し、ガンマ関数の大域的部分積分多様体と Jones 多項式の量子テレポーテーションという思考実験を可視化しながら送信し、さらに音声帯域の FSK 変調によってアプリとスマートフォンの間で**実際に**文字データを送受信する。

一方、BadaGPT 系列（BadaGPT道場、BadaGPT改）では、対話型 AI がアプリケーションを作るときの技法を Bada の上で扱ってきた。本研究はこの二つを統合し、「電信で送られた依頼（思考入力）を受け取っ

た BadaGPT が、依頼の全体を捉えてアプリケーションを錬成する」システムを構成する。

1.2 課題

利用者が頭の中のアプリケーションを言葉にすると、その文章には機能、扱うデータ、項目、並べ替えや検索といった操作、配色、利用する端末などの要求が一文の中に混在する。生成系がこのうち目立つ部分だけを拾い、残りを黙って捨てると、利用者は「言ったのに作られていない」部分を成果物の中から探すことになる。生成系が**何を作り、何を作らなかったか**が見えないことが問題である。

1.3 本研究の貢献

1. 依頼文の全節を仕様要素へ対応付け、未解釈の節を成果物と論文に保存する「全体把握」の手続きと、その網羅率の定義（第 4 章）。
2. 互いに独立した 14 種の実行時部品を組み合わせて依存ゼロの単一 HTML アプリを生成し、機械的に検証する生成器（第 5 章）。
3. 生成のたびに要求・設計・実装・検証を論文の体裁で書き出し、APK / Windows / Linux のビルド一式とともに配布する仕組み（第 6 章）。
4. 開発用と未見用の二つの依頼集合による定量評価と、実ブラウザ・実パッケージでの動作検証（第 7 章）。

---

2. 関連する取り組み

BADA グラビトン電信（bada_telegraph/） ― Bada インタプリタ（set / <- / -< / >- / Omega::push / print / Tele::）、シャノンエントロピー H とベータ関数による不変量 三 = β(H+1, M+1)/log(N+2)、Kauffman 括弧による (2,n) トーラス結び目の Jones 多項式、モールス符号、FSK 音響モデムを持つ。本研究はこれらをそのまま電信層として用いる。

BadaGPT道場（bada_gpt_dojo/） ― トークン化・自己注意・次トークン予測・意図解析・計画と生成・検証と自己修正の 7 つの技を教えるアプリ。意図解析とアプリ錬成を持つが、依頼文のどの部分を何に使ったかは記録しない。

BadaGPT改（bada_gpt_kai/） ― 文章・音声・スケッチ・写真から Claude を用いて APK / EXE 用プロジェクトや PDF を生成する。生成の主体は大規模言語モデルである。

要求工学におけるトレーサビリティ ― 要求と設計・実装の対応を追跡可能にするという考え方は古くからある。本研究の網羅率は、これを依頼文の節という細かい単位で、生成のたびに自動で行うものと位置付けられる。

---

3. システム構成

3.1 パイプライン

BadaGPT電信の処理は、Bada プログラム badagpt.bada の実行として表される。

```
set entropy   = H            # 依頼文のシャノンエントロピー（送信時に注入）
set crossings = n            # Jones ブレイドの交差数（送信時に注入）
line <- "thought request telegraph"
line -< 1.0
line >- line
Omega::push line as thought_node
Tele::encode                 # モールス + エントロピー符号化
Tele::knot crossings         # Jones 多項式で量子チャンネルの結び目
Tele::teleport               # 量子テレポーテーション（思考実験）
Tele::graviton               # グラビトン → 電磁波 搬送波（思考実験）
Tele::send                   # BadaGPT へ配信
GPT::comprehend              # 全体把握
GPT::plan                    # 設計
GPT::generate                # 生成
GPT::verify                  # 検証
GPT::paper                   # 論文
```

インタプリタは `Tele::` と `GPT::` の名前空間命令を受け取るたびに画面の対応する段を駆動する。`Tele::` 段の数値は山口フレームワークに基づく思考実験の可視化であり、生成結果の正しさには用いない。`GPT::` 段が本研究の主題である。

3.2 思考入力

本稿でいう「思考入力」とは、利用者が思い描いたアプリケーションを（a）文章で書く、（b）音声認識で話す（対応環境のみ）、（c）別の端末から FSK 音で電信する、のいずれかで与えた依頼文を指す。脳の信号を読み取る機能はない。

（c）の FSK 電信は、space 1000 Hz / mark 1600 Hz・100 baud の UART 型フレーム（プリアンブル、SOF 0x7E、2 バイト長、UTF-8 本文、チェックサム）で、Goertzel 法により復調する。スマートフォンで書いた依頼をパソコンのアプリへ音で送ることができ、電話のスピーカー越しでも届く。

3.3 実装の構成

ファイル	行数	役割
src/engine.js	435	ZIP / PDF / Markdown、Bada インタプリタ、ミ、Jones、モールス、FSK
src/comprehend.js	320	全体把握 (節分割・対応付け・網羅率)
src/silent.js	471	サイレント・トーク (ジェスチャー / 非言語アトム・トランスフォーマー・latent → 文章 / 画像 / Bada)
src/generate.js	779	14 種の実行時部品、HTML 組み立て、検証
src/paper.js	435	論文の自動執筆、ビルド一式、Claude (任意)
src/ui.js	545	画面
tools/engine-test.js	286	単体テスト 132 件
tools/evaluate.js	114	第 7 章の評価の再現

tools/build.js がこれらを 1 枚の index.html に埋め込み、同じファイルを Cordova（Android）と Electron（Windows / Linux）に同梱する。

---

4. 全体把握

4.1 節への分割

依頼文を句点・感嘆符・疑問符・改行で文に分け、さらに読点で節に分ける。ただし項目の列挙を含む文（「項目はタイトル、著者、読了日」「日付、体重、運動時間、メモ」）は読点で切らずに 1 節として保つ。列挙文の判定は、(i)「項目・フィールド・カラム」等の語を含む、(ii) Rails 形式 name:type を含む、(iii) 区切りが 2 個以上で「を/の + 記録・管理・登録…」で終わる、(iv) 区切りが 2 個以上で各要素が短い名詞だけからなる、のいずれかとした。

4.2 仕様要素への対応付け

各節を次の規則に照合する。

仕様要素	規則の例
モジュール (14 種)	「家計簿・支出・収入」→ 家計簿、「タイマー・ポモドーロ」→ タイマー、「量子・ベル状態」→ 量子回路 など
記録対象	「X を管理/記録/登録」「X の Y 記録」「X 管理」の X (末尾の漢字・カタカナ・英数字の連続)
項目と型	列挙文の各要素。「日・期限」→ 日付、「評価・点数」→ 評価 (1–5)、「金額・数量」→ 数値、「メモ・感想」→ 長文
機能	検索・並べ替え・CSV 書き出し・グラフ・音・印刷
見た目	ダーク / ライト、9 色のアクセント色
対象環境	スマホ・APK → Android、Windows・パソコン → Windows、Linux・Ubuntu → Linux
題名・目的	「」で囲まれた名前、「〜したい」「〜のため」の節
作成依頼の意図	「作って」「欲しい」「お願い」などの短い節

記録対象が既存モジュールを指す場合（「タスクを管理」→ ToDo、「支出を記録」→ 家計簿）は、汎用の記録モジュールを作らずにそのモジュールへ回す。題名の「」の中の語や項目の列挙の語でモジュールを呼ばないよう、それらを除いた本文でモジュールを判定する。

4.3 網羅率と、捨てないこと

少なくとも一つの仕様要素に対応した節を「網羅された節」とし、

> 網羅率 = 網羅された節の数 / 全節の数

と定義する。網羅されなかった節は**未解釈の節**として、(1) 生成アプリの「この依頼について」画面に要望メモとして、(2) 論文の考察に原文のまま、(3) Claude を使う場合はプロンプトに UNRESOLVED とし

て、必ず残す。依頼に明示されなかった点（項目が無い、対象環境の指定が無い等）を補った場合は、その**仮定**も同じ三箇所に記す。

---

5. 生成と検証

5.1 実行時部品

部品	主な機能
記録・管理	任意の項目と型による登録・編集・削除・一覧、検索、並べ替え、CSV / JSON、グラフ
ToDo	期限・優先度、完了、絞り込み、進捗バー
メモ	一覧と本文、自動保存、検索、テキスト書き出し
家計簿	収入 / 支出・分類・金額、月ごとの集計と分類別グラフ、CSV
タイマー	ストップウォッチ (ラップ)、カウントダウン、ポモドーロ、アラーム音
電卓	eval を使わない再帰下降構文解析、履歴
カウンター	複数のカウンター
クイズ	単語カードの追加、シャッフル出題、採点
お絵かき	ペン / 消しゴム、色・太さ、元に戻す、PNG 保存
単位変換	長さ・重さ・面積・温度
抽選	サイコロ、コイン、くじ引き
習慣	直近 14 日の記録と連続日数
電信	モールス符号化・復号・打電音
量子回路	2 qubit 状態ベクトル、H / X / Z / CNOT、ベル状態、1000 回測定の分布

各部品は生成器のソースの中に**本物の JavaScript 関数**として書かれており、`Function.prototype.toString` で文字列化して生成アプリへ埋め込む。そのため部品のコードは生成器自身の構文検査とテストの対象になる。設定値は `<` を `<` に置き換えた JSON として埋め込み、利用者の文字列で `script` 要素が途切れないようにした。データは端末内（localStorage）にのみ保存し、ネットワーク通信は行わない。

5.2 検証

生成のたびに次を機械的に検査する: HTML 骨格、JavaScript 構文、外部読み込みの有無（オフライン性）、依頼されたモジュールの実装、項目（データモデル）の実装、題名の反映、網羅率（50% 以上）、サイズ。さらに画面上では生成アプリをサンドボックス化した iframe で実際に起動し、読み込み中の実行時エラーを親画面へ通知させて記録する。

5.3 Claude による生成 (任意)

API キーが設定されている場合、全体把握の結果（未解釈の節を含む）と原文を Claude に渡し、単一 HTML の生成を依頼する。既定のモデルは Claude Opus 5.5、思考は adaptive、拒否時は既定の代替モデ

ルへ回す。返った HTML は同じ検証にかけ、不合格なら内蔵エンジンの結果を用いる。キーが無ければすべてオフラインで動作する。

---

6. 論文の自動執筆と配布

GPT::paper は、生成ごとに次の章立ての論文を Markdown で書き出す: 要旨、序論（依頼の原文）、方法（電信指標・全体把握・生成と検証）、結果（節ごとの対応表、モジュールと根拠、データモデル、実装の構成、検証結果）、考察（仮定、未解釈の節、限界）、結論、参考文献。数値はすべて実際の解析・生成・検証の結果から埋める。

この論文は画面で表示でき、Markdown または PDF として保存できる。PDF は日本語を端末のフォントで A4 の画像ページに描き、自前の PDF 書き出し器で埋め込む。

さらに「ビルド一式」として、生成アプリ・論文・Electron 設定・Cordova 設定・GitHub Actions ワークフローを ZIP にまとめる。この ZIP をリポジトリに置いて実行すれば、**生成したアプリそのもの**が Android APK・Windows EXE・Linux AppImage / deb になる。BadaGPT電信自身も同じ方式で 3 種のプラットフォームへビルドし、GitHub Release に添付する。

6.5 拡張:サイレント･トーク (非言語思考入力)

言葉を介さずに依頼を与える拡張を加えた。中心となる入力は、利用者が頭に**思い描いたもの**を、文字や絵文字を選ぶのではなく**連続した動き**（ジェスチャー）で面になぞることである。その動きの速さ・広がり（経路長）・向き（8 方位ヒストグラム）・曲がり（角度変化の累積）・リズム（ストローク間隔の分散）・重心を固定の規則で特徴量にし、そこから非言語のアトム列を導く。重心の 3×3 領域が主モジュールを、経路長が文章の分量を、曲がりが気分と配色を、重心の高さが明暗を、主な向きが機能を決めるというように、動きの何がどの意味になったかを「読み取った内容」として利用者にそのまま示す。細かく補うために**無言のグリフ**（感情・領域・対象・動作・分量・環境の 33 アトム）も足せる。

この非言語の列を、固定の重みを持つ小さな**トランスフォーマー**（埋め込み 16 次元・自己注意 2 層・4 頭・位置エンコードつき）が、言語化されていない潜在表現（latent）に変換する。重みは固定の疑似乱数で初期化され学習は行わないため、同じ動きは必ず同じ latent を与える（決定的）。自己注意の各行は確率分布になる（行和 = 1、実測誤差 2×10^{-16}）。

latent からは四つを決定的に復号する: (a) **本と同じ分量まで展開できる文章**（分量アトムに応じて素描〜約 96,000 字。latent と疑似乱数で駆動するテンプレート展開）、(b) **イメージ**（latent から生成する SVG）、(c) **Bada 言語のプログラム** silent.bada（BadaInterp が構文エラーなく実行できる）、(d) 以降の全体把握へ渡す**依頼文**。

正直な範囲として、これは利用者が**自分で描いた動き**を固定の規則で言葉にするものであり、脳の信号は読み取らない。トランスフォーマーは学習済みの大規模言語モデルではなく、文章は意味のある新規の内容ではなく latent に駆動された展開である。単体テストで、決定性・注意行和・分量の増加・Bada プログラムの実行可能性・下流の全体把握との接続を検証した（第 7.1 節）。

---

7. 評価

7.1 単体テスト

tools/engine-test.js の 132 件（サイレント・トークのトランスフォーマー、Bada インタプリタ、三・Jones・モールス、FSK の往復・雑音下・256 バイト超、全体把握、14 部品の単独生成と全部入り、</script> を含む題名、検証器の検出能力、論文の章立て、ZIP / PDF の構造、Claude リクエストと SSE 解析）はすべて合格した。

7.2 開発用の依頼集合 (24 件)

規則を作りながら用いた 24 件の依頼に、人手でモジュールと項目の正解を付けた。最初の評価では F1 92.5%、完全一致率 87.5% であり、次の 3 種の誤りを見つけて一般的な規則として修正した:（a）名詞だけを並べた文を項目の列挙と認識できず「メモ」でメモ部品を誤って呼ぶ、（b）「映画の鑑賞記録」のような複合名詞から記録対象を取り出せない、（c）「締め切り・チェックリストの管理」を ToDo に回せない。修正後の結果を示す。この集合は規則の開発に使ったため、次節の未見集合より楽観的な値になる。

指標	値
依頼文の数	24
モジュール選択の適合率 / 再現率 / F1	100.0% / 100.0% / 100.0%
モジュール集合の完全一致率	100.0%
項目抽出の完全一致率 (7 件)	100.0%
節の網羅率 (平均 / 最小)	100.0% / 100.0%
生成アプリの静的検証合格率	100.0%

7.3 未見の依頼集合 (10 件)

規則の修正を終えた後に新たに 10 件の依頼を書き、**規則を一切変えずに一度だけ**評価した。

#	依頼文	正解	選択	一致	項目
1	レシピを管理するアプリ。料理名、材料、手順、調理時間を記録したい。	records	records	○	○
2	会議の時間を計るストップウォッチ。	timer	timer	○	—
3	旅行の支出を記録して合計を見たい。	budget	budget	○	—
4	植物の水やりを管理したい。	records	notes+todo	×	×
5	歴史の年号を覚えるための問題集。	quiz	quiz	○	—
6	毎日の筋トレの継続を記録する習慣アプリ。	habit	records+habit	×	—
7	ボードゲーム用のサイコロとタイマーをひとつに。	random+timer	timer+random	○	—
8	アイデアを書き留めるノートと、やることリスト。	notes+todo	records+todo+notes	×	—
9	社員名簿を作りたい。氏名、部署、内線番号、入社日。	records	records	○	○
10	ベル状態を作って測定する量子の練習。	quantum	quantum	○	—

指標	値
モジュール選択の適合率 / 再現率 / F1	73.3% / 91.7% / 81.5%
モジュール集合の完全一致率	70.0%
項目抽出の完全一致率 (3 件)	66.7%
節の網羅率 (平均 / 最小)	100.0% / 100.0%
生成アプリの静的検証合格率	100.0%

誤りの分析. #4 は「水やり」の末尾がひらがなのため記録対象を取り出せず、何も読み取れなかった場合の既定（メモ + ToDo）に落ちた。#6 は「習慣」の部品で足りるのに「記録する」から汎用の記録部品を重ねた。#8 は「やることリスト」の「リスト」を記録対象と見なし、記録部品を余分に加えた。いずれも**余分な部品を加える**方向の誤りであり、要求を落とす誤りは #4 の 1 件（再現率 91.7%）であった。

網羅率についての知見. 誤った 3 件でも網羅率は 100% であった。網羅率は「どの節も何かに対応付けられ、黙って捨てられていない」ことを保証するが、「その対応付けが正しい」ことは保証しない。だからこそ本システムは、節ごとの対応表を画面と論文に必ず示し、利用者が対応付けの誤りを目で確かめられるようにしている。

7.4 実環境での動作確認

- **実ブラウザ (Chromium)**: 7 件の依頼から BadaGPT電信の画面操作でアプリを生成し、生成アプリの全タブについて入力欄への入力、選択肢の変更、すべてのボタン、チェックボックス、キャンバスへの描画を自動操作した。全 7 件でパイプライン 10 段がすべて完了し、検証はすべて合格、実行時エラーは 0 件であった。iframe 内で意図的に投げた例外が捕捉されることを別途確かめ、この 0 件が見逃しでないことを確認した。HTML・ZIP・PDF・Markdown の保存も確認し、ZIP は 7 ファイルで整合性検査に合格、PDF は A4 で 4 ページであった。

- **パッケージ (Linux)**: Electron で AppImage と deb を実際にビルドし、パッケージ化したアプリを仮想ディスプレイ上で起動して、同梱の `resources/www/index.html` から全パイプラインが完了し全検証に合格することを確認した。
- **Android APK / Windows EXE**: 同じ `index.html` を同梱する Cordova / Electron の設定と GitHub Actions ワークフローで構築する。本リポジトリの BadaGPT道場で実績のある手順（cordova-android 12・Gradle 7.6.4 固定、electron-builder の NSIS / portable）を踏襲した。

---

8. 考察と限界

規則による解析の限界. 内蔵エンジンは語彙と規則に基づくため、辞書にない新しい種類の機能は 14 部品の組み合わせでしか表せない。未見集合で F1 が 100% から 81.5% に下がったことは、開発集合に合わせて規則を育てると楽観的な値が出ることを示しており、今後も評価には未見の依頼を使い続ける必要がある。誤りの多くは余分な部品を加える方向であり、利用者が「この依頼について」と対応表で気付ける形で現れる。

Claude との分担. API キーを設定した場合、規則による全体把握の結果を Claude に渡すことで、辞書にない機能も生成できる。このとき規則側の役割は「どの節をまだ解釈していないか」を明示して取りこぼしを防ぐことに移る。

電信の物理量. グラビトン⇔電磁波変換と量子テレポーテーションの数値は山口フレームワークに基づく思考実験の可視化であり、物理的な通信を主張するものではない。実際に動作する通信は FSK 音響電信である。

思考入力. 本システムは文章・音声・電信で与えた依頼文を扱い、脳の信号は読み取らない。

---

9. 結論

BADA グラビトン電信の電信エンジンの上に、依頼の全節を仕様へ対応付ける全体把握、14 部品による自己完結アプリの生成、機械的な検証、生成過程の論文化、3 プラットフォームへの配布を備えた BadaGPT 電信を構成した。未見の依頼で F1 81.5% という結果は規則による解析の限界を示す一方、網羅率と節ごとの対応表は、生成系が何を作り何を作らなかったかを利用者に見えるようにした。未解釈の節と仮定を捨てずに残すという設計は、生成方式（規則か大規模言語モデルか）によらず有効であると考える。

謝辞

本システムの実装と本稿の作成には、Anthropic の Claude（Claude Code）の支援を受けた。

参考文献

1. 山口 雅旭. Bada 言語と大域的部分積分多様体. masaaki-avnturle/Bada.
2. 山口 雅旭. BADA グラビトン電信 — Jones 多項式の量子テレポーテーションと FSK 音響通信. masaaki-avnturle/Bada (`bada_telegraph/`).

3. 山口 雅旭. BadaGPT道場 / BadaGPT改. masaaki-avnturle/Bada (bada_gpt_dojo/, bada_gpt_kai/).
4. V. F. R. Jones. A polynomial invariant for knots via von Neumann algebras. *Bulletin of the AMS* 12 (1985) 103-111.
5. L. H. Kauffman. State models and the Jones polynomial. *Topology* 26 (1987) 395-407.
6. C. E. Shannon. A Mathematical Theory of Communication. *Bell System Technical Journal* 27 (1948) 379-423, 623-656.
7. G. Goertzel. An Algorithm for the Evaluation of Finite Trigonometric Series. *American Mathematical Monthly* 65 (1958) 34-35.
8. O. C. Z. Gotel, A. C. W. Finkelstein. An Analysis of the Requirements Traceability Problem. *Proc. IEEE ICRE* (1994) 94-101.

付録 A. 再現方法

```
node bada_gpt_telegraph/tools/build.js              # src/ → index.html
node bada_gpt_telegraph/tools/engine-test.js        # 単体テスト 132 件
node bada_gpt_telegraph/tools/evaluate.js           # 7.2 開発用の依頼集合
node bada_gpt_telegraph/tools/evaluate.js --heldout # 7.3 未見の依頼集合
```

アプリ本体は bada_gpt_telegraph/index.html をブラウザで開くだけで動作する。